CSS3122 DATA STRUCTURE AND ALGORITHMS

CH5 ABSTRACT DATA STRUCTURE

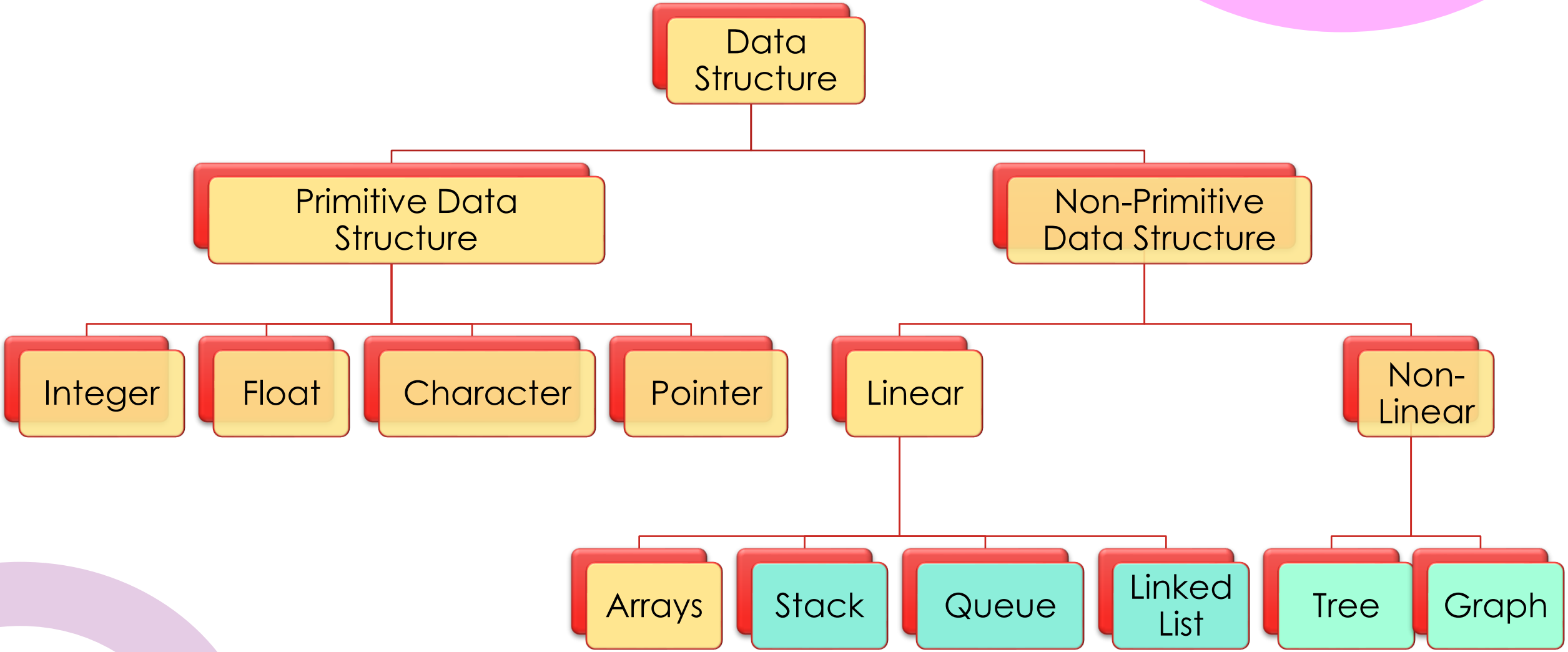
KHAIRUNNISA IBR

CHAPTER OUTLINE

In this chapter, you will learn about:

1. Linked List
2. Stack
3. Queue
4. Tree
5. Graph

INTRODUCTION



LINKED LIST

LET US RECAP ON ARRAY…

- Array is a linear collection of data elements, which are stored in consecutive memory locations.
- The declared array size will restrict the no of elements that can be stored.
- Example:

  int marks[10]; → **can store max 10 int data elements**

- **What if we don't know the no of elements in advance?**
- **How to use memory efficiently where elements are stored randomly instead of consecutively?**

LINKED LIST (CONT…)

LINKED LIST

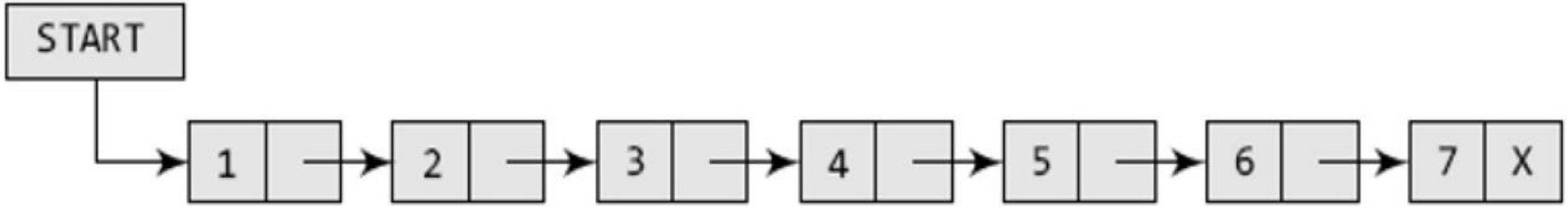
- Is a data structure that is free from the aforementioned restrictions.
- Does not store its elements in consecutive memory locations.
- User can add any number of elements to it.
- But, unlike array, linked list does not allow random access of data.
- Elements can only be accessed sequentially.
- But, like array, insertions & deletions can be done at any point in the list.

LINKED LIST (CONT…)

- Is a linear collection of data elements, called nodes.
- Is a data structure that can implement other data structures.
- Example: it acts as a building block to implement data structures such as stacks, queues, and their variations.
- Can be perceived as a ‘train’ or a sequence of nodes in which each node contains
  - one or more data fields, and
  - a pointer to the next node.

LINKED LIST (CONT…)

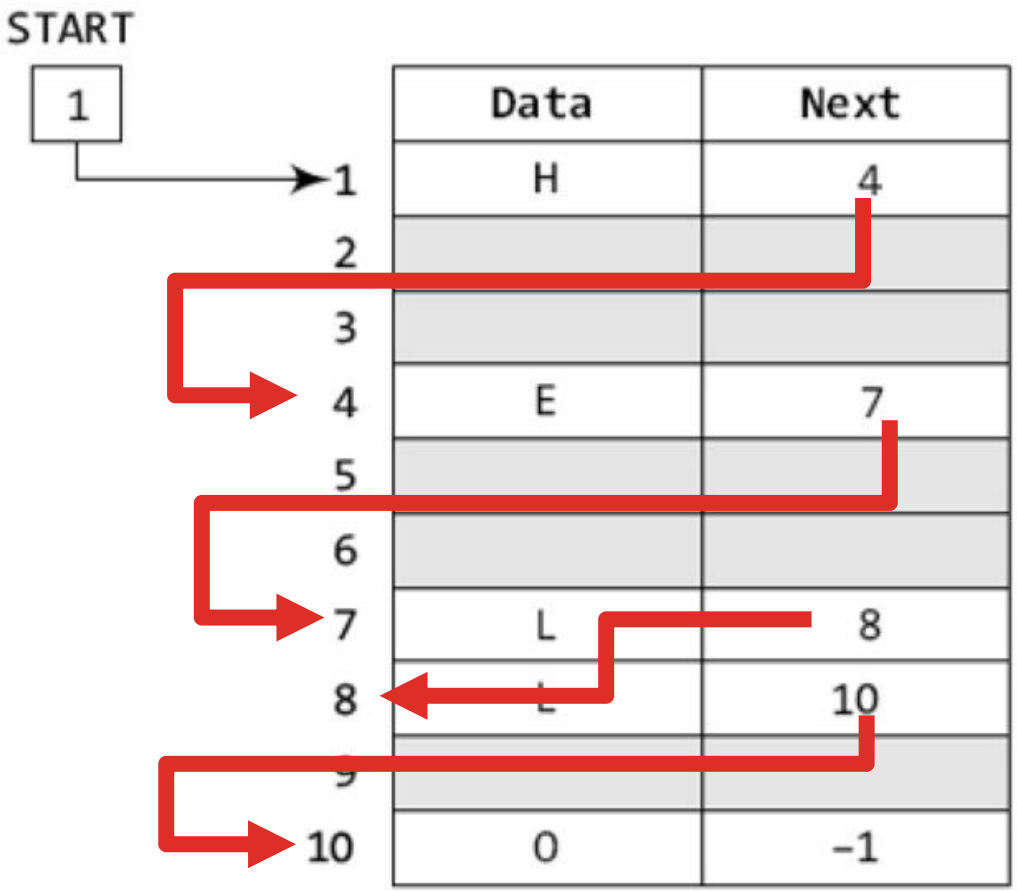
- Every node contains 2 parts:
  1. **An integer** (left part) – contain data
  2. **A pointer to the next node** (right part) – address of the next node in sequence.
- The last node will have no node connected to it.
  - It will store the value NULL.



- In the figure, the NULL pointer is represented by X.
- In code, we define NULL as -1.
- A NULL pointer denotes the end of the list.

LINKED LIST (CONT…)

EXAMPLE 1:



- Each node has 2 fields: DATA and NEXT.
  - DATA – stores information part.
  - NEXT – stores address of next node.
- Variable START stores address of the first node.
- If START = 1, hence, first data is stored at address 1.
- Traversing the linked list will give characters that form the word 'HELLO'.

LINKED LIST (CONT…)

EXAMPLE 2:

START 1

1 (Biology)

2

START 2
(Computer Science)

	Roll No	Next
1	S01	3
2	S02	5
3	S03	8
4		
5	S04	7
6		
7	S05	10
8	S06	11
9		
10	S07	12
11	S08	13
12	S09	-1
13	S10	15
14		
15	S11	-1

2 linked lists in the memory

- 2 different linked list can be simultaneously maintained in the memory.
- There is no ambiguity in traversing through the list because each list has a separate START pointer.

LINKED LIST (CONT…)

EXAMPLE 3:

- A linked list that stores the roll no, name, aggregate & grade of students.
- The NEXT pointer is used to store the data alphabetically.

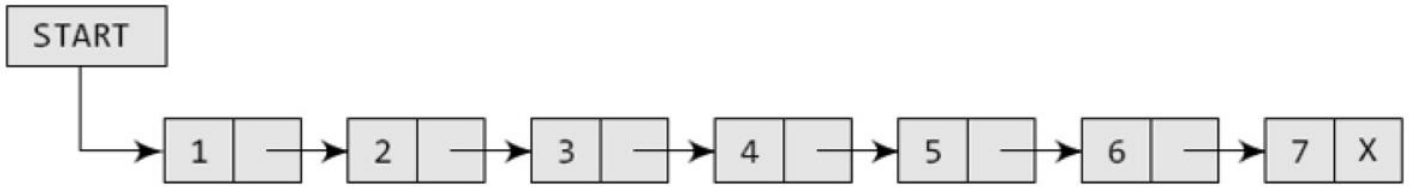
START
18

	Roll No	Name	Aggregate	Grade	Next
1	S01	Ram	78	Distinction	6
2	S02	Shyam	64	First division	14
3					
4	S03	Mohit	89	Outstanding	17
5					
6	S04	Rohit	77	Distinction	2
7	S05	Varun	86	Outstanding	10
8	S06	Karan	65	First division	12
9					
10	S07	Veena	54	Second division	-1
11	S08	Meera	67	First division	4
12	S09	Krish	45	Third division	13
13	S10	Kusum	91	Outstanding	11
14	S11	Silky	72	First division	7
15					
16					
17	S12	Monica	75	Distinction	1
18	S13	Ashish	63	First division	19
19	S14	Gaurav	61	First division	8

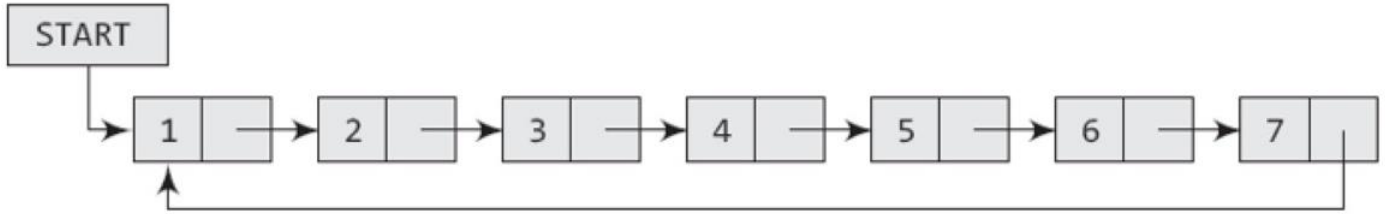
LINKED LIST (CONT…)

LINKED LIST TYPES

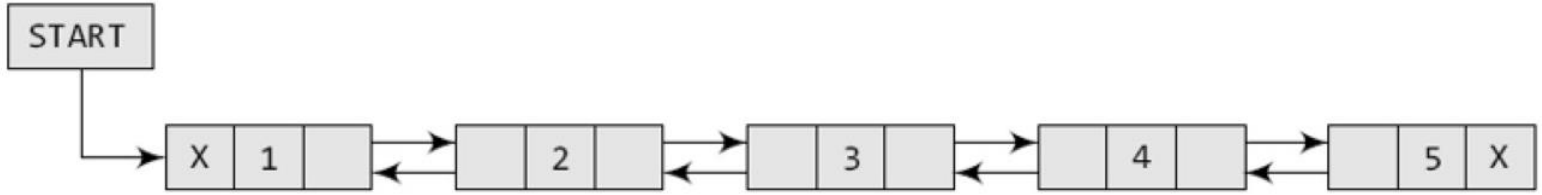
- Single linked lists



- Circular linked lists



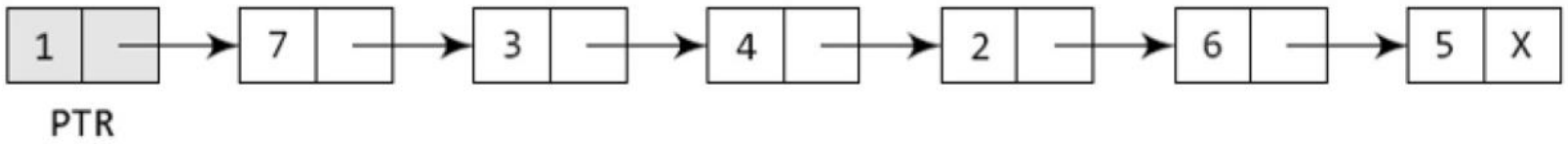
- Doubly linked lists



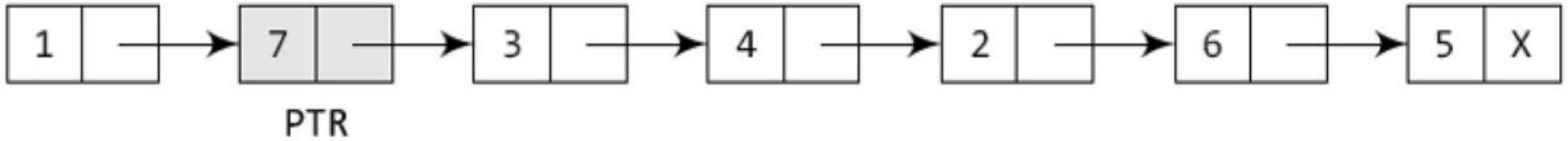
LINKED LIST (CONT…)

Single linked lists

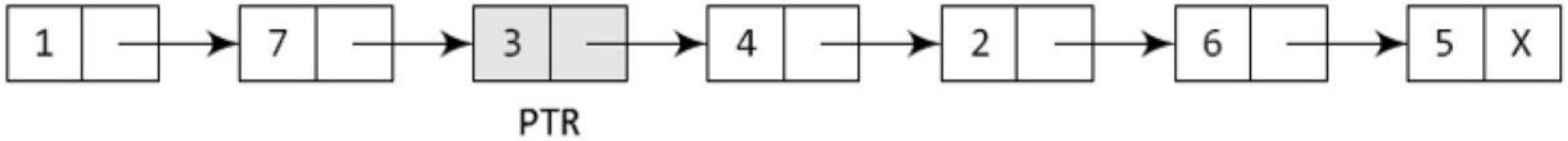
- Searching a linked list:
  - If we are **to looked for VAL=4**, then the searching algorithm works like this:



Here PTR –> DATA = 1. Since PTR –> DATA != 4, we move to the next node.



Here PTR –> DATA = 7. Since PTR –> DATA != 4, we move to the next node.



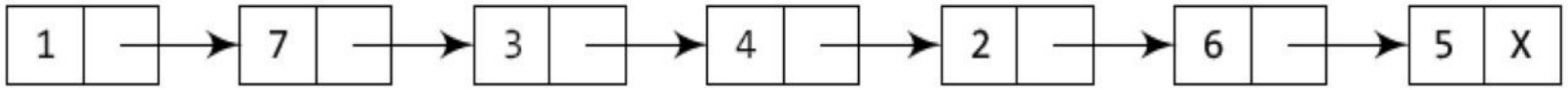
Here PTR –> DATA = 3. Since PTR –> DATA != 4, we move to the next node.

Here PTR –> DATA = 4. Since PTR –> DATA = 4, POS = PTR. POS now stores the address of the node that contains VAL

LINKED LIST (CONT…)

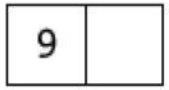
Single linked lists

- Inserting an element **at the beginning** of a linked list:

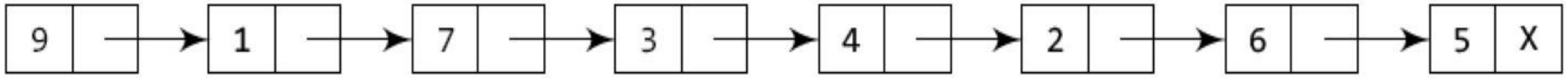


START

Allocate memory for the new node and initialize its DATA part to 9.

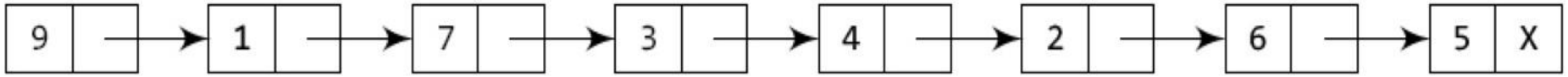


Add the new node as the first node of the list by making the NEXT part of the new node contain the address of START.



START

Now make START to point to the first node of the list.

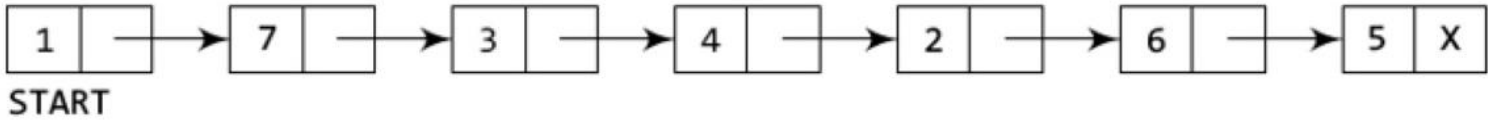


START

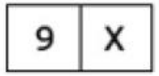
LINKED LIST (CONT…)

Single linked lists

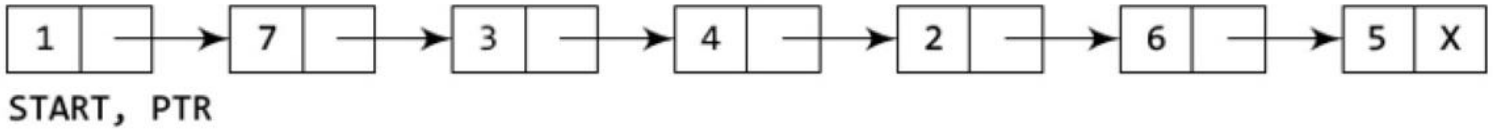
- Inserting an element **at the end** of a linked list:



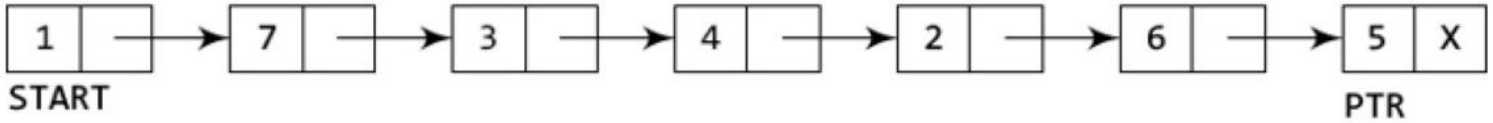
Allocate memory for the new node and initialize its DATA part to 9 and NEXT part to NULL.



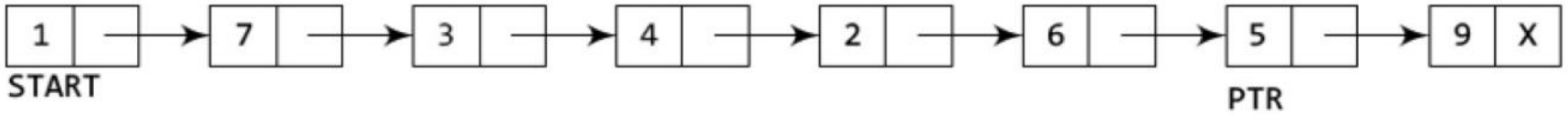
Take a pointer variable PTR which points to START.



Move PTR so that it points to the last node of the list.



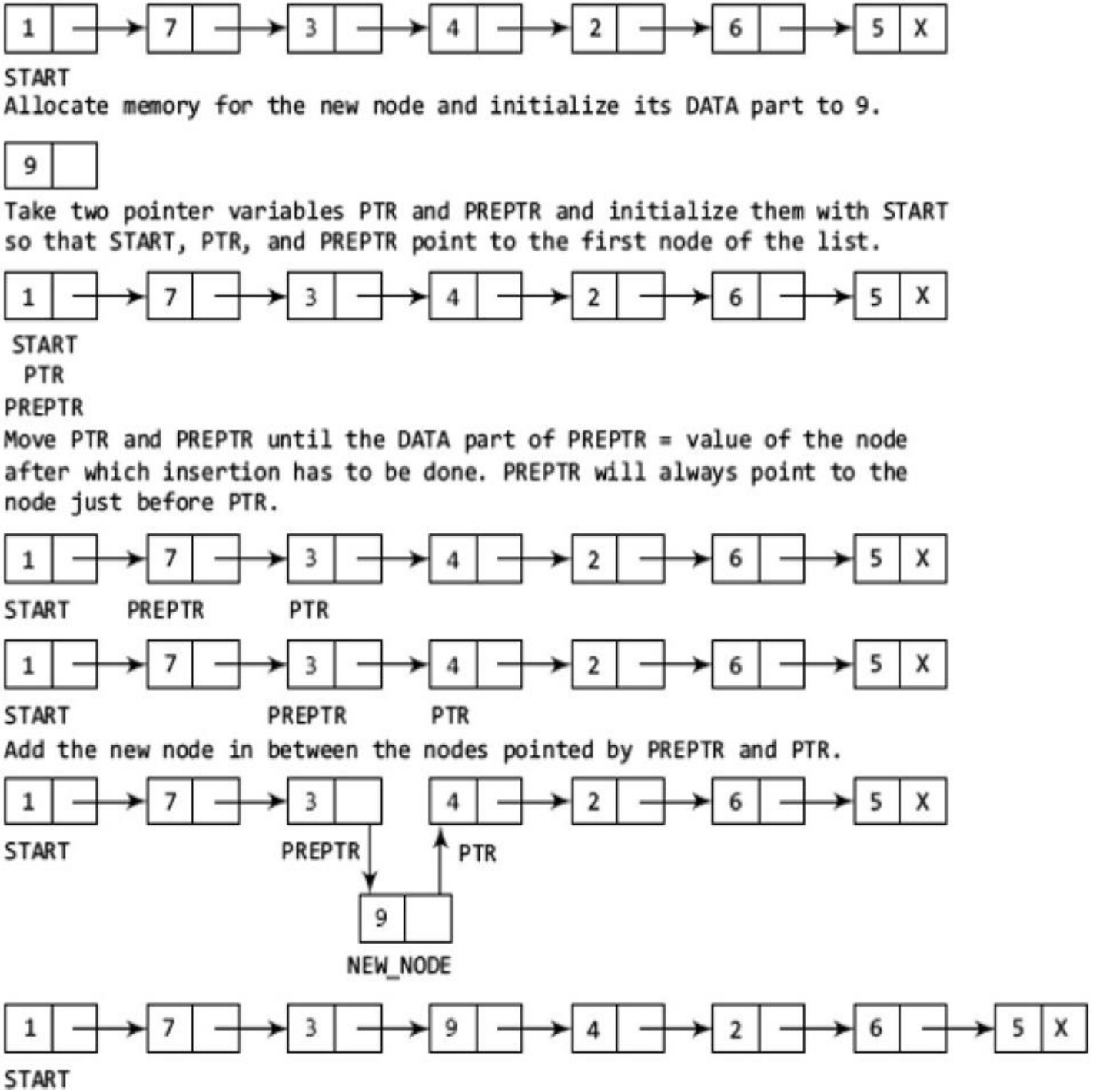
Add the new node after the node pointed by PTR. This is done by storing the address of the new node in the NEXT part of PTR.



LINKED LIST (CONT…)

Single linked lists

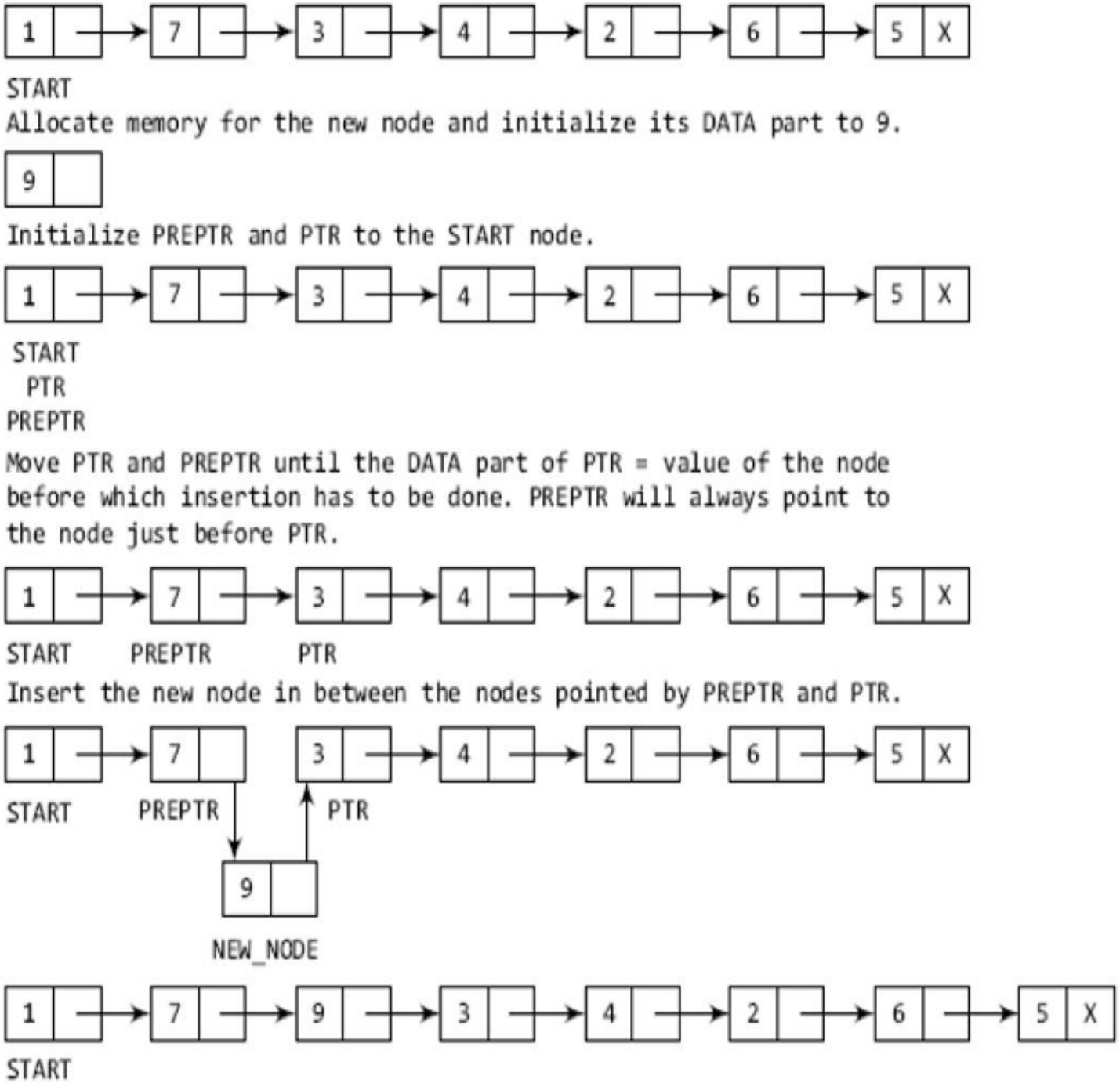
- Inserting an element **after** given a node in a linked list:
- E.g. insert 9 after 3



LINKED LIST (CONT…)

Single linked lists

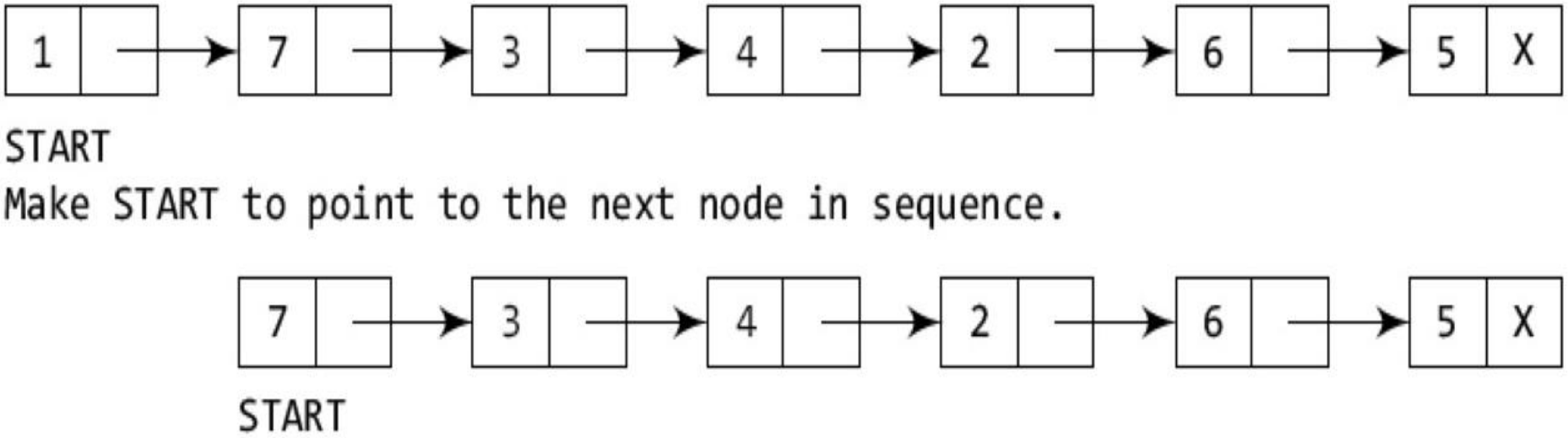
- Inserting an element **before** given a node in a linked list:
- E.g. insert 9 before 3



LINKED LIST (CONT…)

Single linked lists

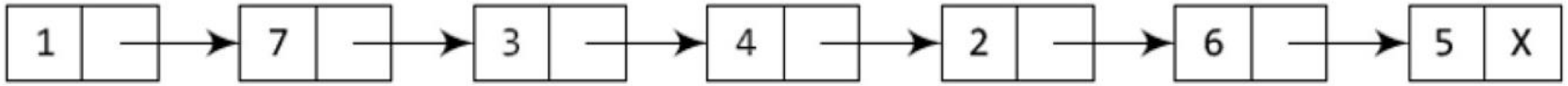
- Deleting the **first node** of a linked list



LINKED LIST (CONT…)

Single linked lists

- Deleting the **last node** of a linked list

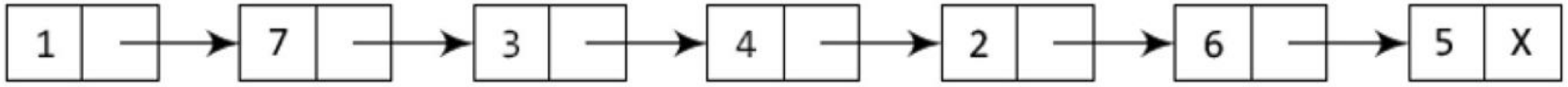


START

Take pointer variables PTR and PREPTR which initially point to START.

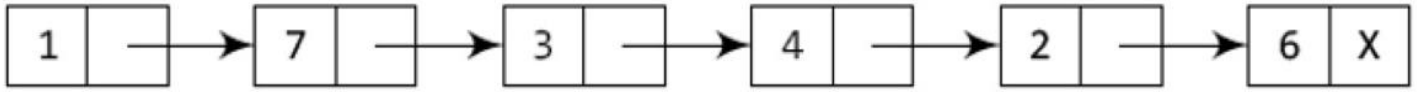
START
PREPTR
PTR

Move PTR and PREPTR such that NEXT part of PTR = NULL. PREPTR always points to the node just before the node pointed by PTR.



START PREPTR PTR

Set the NEXT part of PREPTR node to NULL.

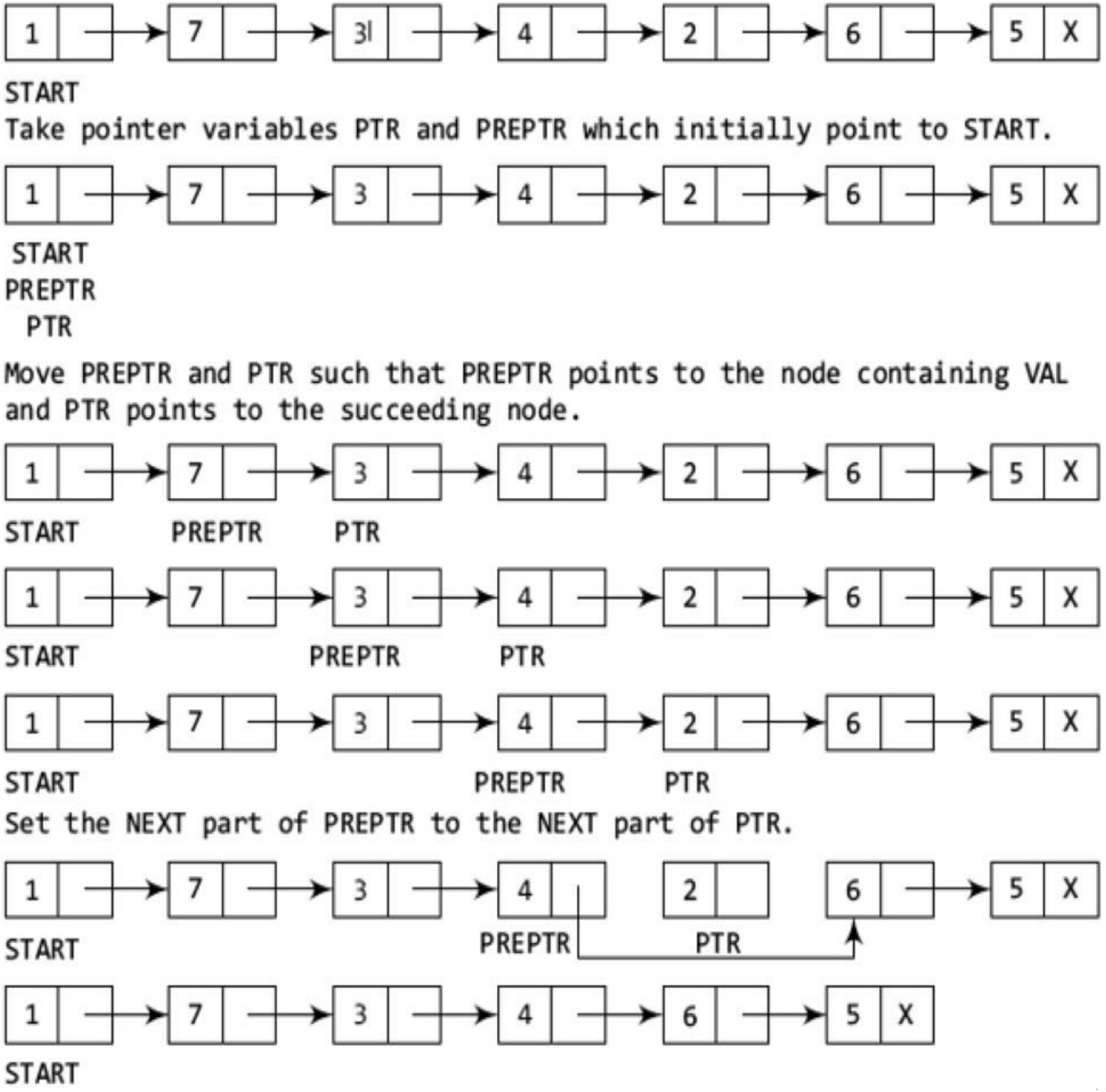


START

LINKED LIST (CONT…)

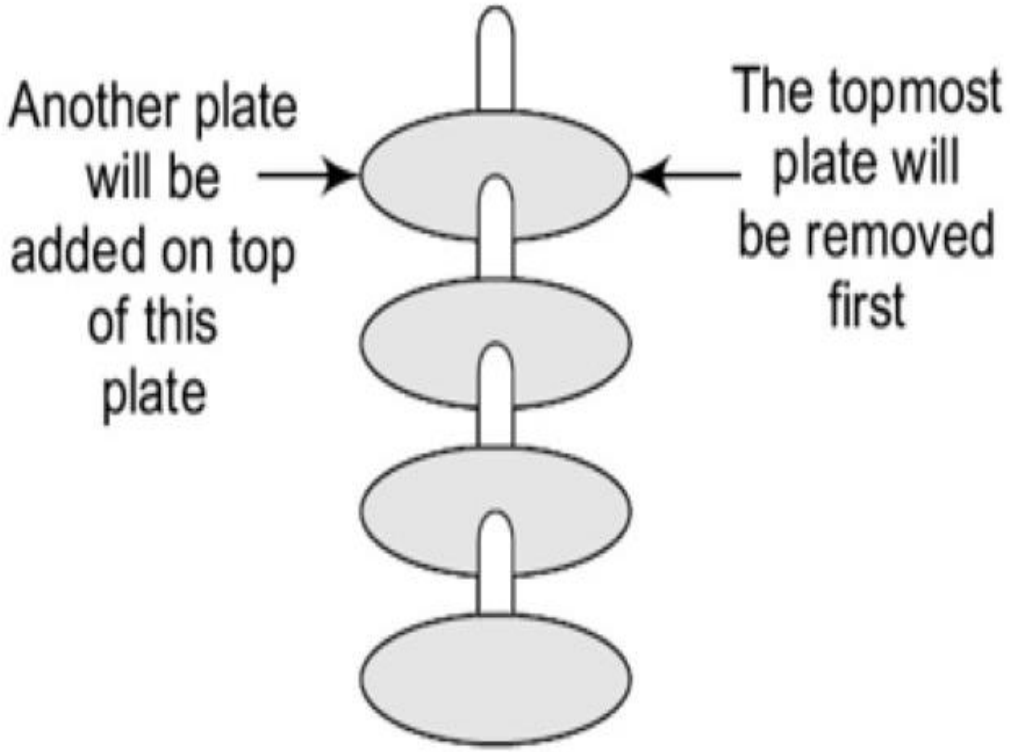
Single linked lists

- Deleting the node **after** a given node in a linked list



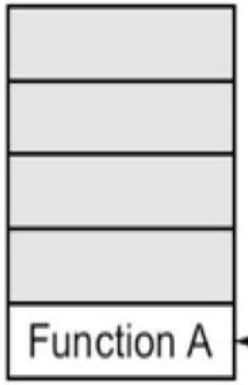
STACK

- A data structure which stores its elements in an ordered manner.
- When you remove a plate from a pile of plates, you remove the topmost plate first.
- Similar case with stack.
- Elements are added and removed only from one end, called the TOP.
- A stack is a LIFO data structure (**L**ast-**I**n-**F**irst-**O**ut), as the last element inserted is the first one to be taken out.

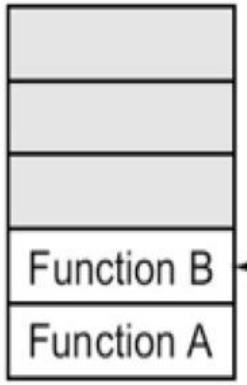


STACK (CONT…)

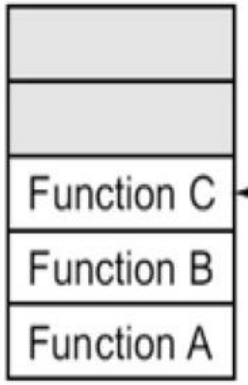
- Where do we need stacks in computer science?
- The answer is in function calls.
- System stack in the case of function calls:



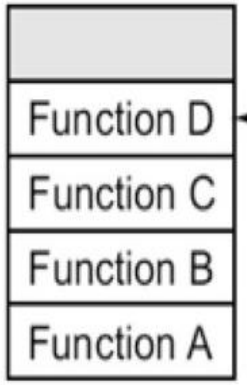
When A calls B, A is pushed on top of the system stack. When the execution of B is complete, the system control will remove A from the stack and continue with its execution.



When B calls C, B is pushed on top of the system stack. When the execution of C is complete, the system control will remove B from the stack and continue with its execution.



When C calls D, C is pushed on top of the system stack. When the execution of D is complete, the system control will remove C from the stack and continue with its execution.



When D calls E, D is pushed on top of the system stack. When the execution of E is complete, the system control will remove D from the stack and continue with its execution.

STACK (CONT…)

STACK OPERATION

- Stack supports 3 basic operations:
  1. **Push operation** – adds an element to the top of the stack.
  2. **Pop operation** – removes the element from the top of the stack.
  3. **Peek operation** – returns the value of the topmost element of the stack.

STACK (CONT…)

PUSH OPERATION

To insert an element with value 6, we first check if `TOP=MAX-1`. If the condition is false, then we increment the value of `TOP` and store the new element at the position given by `stack[TOP]`. Thus, the updated stack becomes as shown in Fig. 7.6.

```
Step 1: IF TOP = MAX-1
            PRINT "OVERFLOW"
            Goto Step 4
        [END OF IF]
Step 2: SET TOP = TOP + 1
Step 3: SET STACK[TOP] = VALUE
Step 4: END
```

Figure 7.7 Algorithm to insert an element in a stack

1	2	3	4	5					
0	1	2	3	**TOP = 4**	5	6	7	8	9

Figure 7.5 Stack

1	2	3	4	5	6				
0	1	2	3	4	**TOP = 5**	6	7	8	9

Figure 7.6 Stack after insertion

STACK (CONT…)

POP OPERATION

To delete the topmost element, we first check if TOP=NULL. If the condition is false, then we decrement the value pointed by TOP. Thus, the updated stack becomes as shown in Fig. 7.9.

```
Step 1: IF TOP = NULL
            PRINT "UNDERFLOW"
            Goto Step 4
        [END OF IF]
Step 2: SET VAL = STACK[TOP]
Step 3: SET TOP = TOP - 1
Step 4: END
```

Figure 7.10 Algorithm to delete an element from a stack

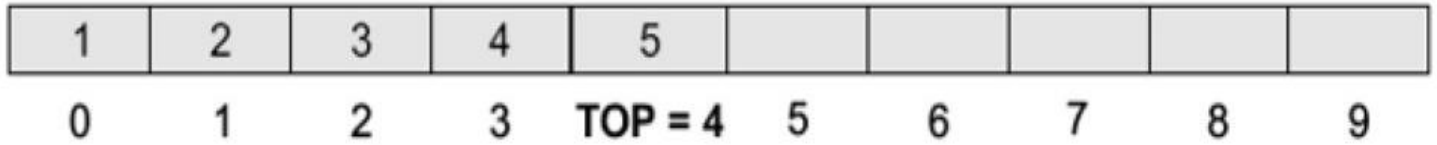


Figure 7.8 Stack

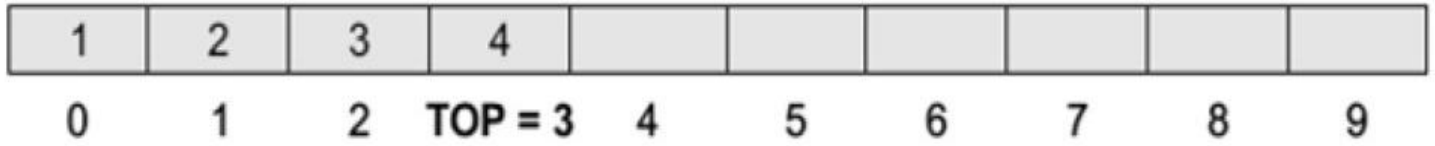


Figure 7.9 Stack after deletion

STACK (CONT…)

PEEK OPERATION

```
Step 1: IF TOP = NULL
            PRINT "STACK IS EMPTY"
            Goto Step 3
Step 2: RETURN STACK[TOP]
Step 3: END
```

Figure 7.11 Algorithm for Peek operation

Peek is an operation that returns the value of the topmost element of the stack without deleting it from the stack. The algorithm for Peek operation is given in Fig. 7.11.

However, the Peek operation first checks if the stack is empty, i.e., if TOP = NULL, then an appropriate message is printed, else the value is returned. Consider the stack given in Fig. 7.12.

1	2	3	4	5					
0	1	2	3	TOP = 4	5	6	7	8	9

Figure 7.12 Stack

Here, the Peek operation will return 5, as it is the value of the topmost element of the stack.

STACK (CONT…)

Typical problems where stacks can be applied:

- Reversing a list
- Parentheses checker
- Conversion and evaluation of arithmetic expressions
  - Conversion of an infix expression into a postfix expression
  - Conversion of an infix expression into a prefix expression
  - Evaluation of a postfix expression
  - Evaluation of a prefix expression
- Recursion
- Tower of Hanoi

STACK (CONT…)

- Reversing a list
  - Reversed by reading each number from an array starting from the first index and pushing it on a stack.

- Parentheses checker
  - Used to check the validity of parentheses in any algebraic expression

(A+B} is invalid. {A+(B-C)} is valid.

- Conversion and evaluation of arithmetic expressions
  - Infix, postfix and prefix notations are 3 different but equivalent notations of writing algebraic expressions.

STACK (CONT…)

- Infix
  - Operator is placed in between the operands.
  - E.g. A+B; here plus operator is placed between the 2 operands A and B.

- Postfix
  - The operator is placed after the operands.
  - E.g. A+B in infix notation, as AB+ is in postfix notation.

- Prefix
  - The operator is placed before the operands.
  - E.g. A+B in infix notation, as +AB in prefix notation.

STACK (CONT…)

Postfix:

- Points to remember before conversion.
- Follow the priority.

Priority No	Operators
1	Brackets ()
2	Exponent ^ , $, ↑
3	Multiplication * , Division /
4	Addition + , Subtraction -

STACK (CONT…)

Postfix:

- Points to remember for stack operations:

Case in stack	Operation	Example
Stack is empty	PUSH(operators)	Empty ← operator
Operator having higher priority	PUSH(operators)	+ ← /
Operator having same priority	POP(operators) PUSH(new operator)	*← / + ← -
Operator having low priority	POP(operators) PUSH(new operator)	* ← + / ← +
No more operands	POP(operators)	----

STACK (CONT…)

- **Postfix :**

Example 7.1 Convert the following infix expressions into postfix expressions.

Solution

```
(a) (A-B) * (C+D)
    [AB-] * [CD+]
    AB-CD+*
(b) (A + B) / (C + D) - (D * E)
    [AB+] / [CD+] - [DE*]
    [AB+CD+/] - [DE*]
    AB+CD+/DE*-
```

- **Prefix :**

Example 7.2 Convert the following infix expressions into prefix expressions.

Solution

```
(a) (A + B) * C
    (+AB)*C
    *+ABC
(b) (A-B) * (C+D)
    [-AB] * [+CD]
    *-AB+CD
(c) (A + B) / ( C + D) - ( D * E)
    [+AB] / [+CD] - [*DE]
    [/+AB+CD] - [*DE]
    -/+AB+CD*DE
```

STACK (CONT…)

- Converting Infix to Postfix:

```
Step 1: Add ")" to the end of the infix expression
Step 2: Push "(" on to the stack
Step 3: Repeat until each character in the infix notation is scanned
        IF a "(" is encountered, push it on the stack
        IF an operand (whether a digit or a character) is encountered, add it to the
        postfix expression.
        IF a ")" is encountered, then
          a. Repeatedly pop from stack and add it to the postfix expression until a
             "(" is encountered.
          b. Discard the "(". That is, remove the "(" from stack and do not
             add it to the postfix expression
        IF an operator O is encountered, then
          a. Repeatedly pop from stack and add each operator (popped from the stack) to the
             postfix expression which has the same precedence or a higher precedence than O
          b. Push the operator O to the stack
        [END OF IF]
Step 4: Repeatedly pop from the stack and add it to the postfix expression until the stack is empty
Step 5: EXIT
```

Figure 7.22 Algorithm to convert an infix notation to postfix notation

STACK (CONT…)

- Convert the following infix expression into postfix:

A–(B/C+(D%E*F)/G)*H

=(A–(B/C+(D%E*F)/G)*H)

Infix Character Scanned	Stack	Postfix Expression
	(	
A	(	A
-	(-	A
(	(-(	A
B	(-(	AB
/	(-(/	AB
C	(-(/	ABC
+	(-(+	ABC/

(cont…)

Infix Character Scanned	Stack	Postfix Expression
…	…	…
(	(-(+(	ABC/
D	(-(+(	ABC/D
%	(-(+(%	ABC/D
E	(-(+(%	ABC/DE
*	(-(+(*	ABC/DE%
F	(-(+(*	ABC/DE%F
)	(-(+	ABC/DE%F*
/	(-(+/	ABC/DE%F*
G	(-(+/	ABC/DE%F*G
)	(-	ABC/DE%F*G/+
*	(-*	ABC/DE%F*G/+
H	(-*	ABC/DE%F*G/+H
)		ABC/DE%F*G/+H*-

STACK (CONT…)

- Converting Infix to Prefix:

```
Step 1: Scan each character in the infix
        expression. For this, repeat Steps
        2-8 until the end of infix expression
Step 2: Push the operator into the operator stack,
        operand into the operand stack, and
        ignore all the left parentheses until
        a right parenthesis is encountered
Step 3: Pop operand 2 from operand stack
Step 4: Pop operand 1 from operand stack
Step 5: Pop operator from operator stack
Step 6: Concatenate operator and operand 1
Step 7: Concatenate result with operand 2
Step 8: Push result into the operand stack
Step 9: END
```

Figure 7.24 Algorithm to convert an infix expression into prefix expression

```
Step 1: Reverse the infix string. Note that
        while reversing the string you must
        interchange left and right parentheses.
Step 2: Obtain the postfix expression of the
        infix expression obtained in Step 1.
Step 3: Reverse the postfix expression to get
        the prefix expression
```

Figure 7.25 Algorithm to convert an infix expression into prefix expression

STACK (CONT…)

– For example, given an infix expression

(A – B / C) * (A / K – L)

```
Step 1: Reverse the infix string. Note that
        while reversing the string you must
        interchange left and right parentheses.
Step 2: Obtain the postfix expression of the
        infix expression obtained in Step 1.
Step 3: Reverse the postfix expression to get
        the prefix expression
```

Solution:

- **Step 1 :**

  (L – K / A) * (C / B – A)

- **Step 2 :**

  The expression is: (L – K / A) * (C / B – A)

  Therefore, [L – (K A /)] * [(C B /) – A]

  = [LKA/–] * [CB/A–]

  = L K A / – C B / A – *

- **Step 3 :**

  Prefix expression is : * – A / B C – /A K L